Category A — Homework Assignment

PROBLEM 1 • **Basic**

Ghost Order Validator

Problem Statement

A campus food delivery app aggregates orders from every hostel-block canteen through several intake channels, including an old web widget that still creates order objects with no validation at all — letting “ghost orders” with a missing student name or missing dish name clutter the kitchen's live queue. Design FoodOrder so an invalid order can never be constructed in the first place, then write a batch processor that reads a list of raw {studentName, dishName} attempts and reports how many were accepted and how many were rejected. An order that's already been marked delivered should not silently accept being marked delivered a second time — the kitchen needs to know if that happens, since it usually means two delivery staff double-served the same table.

Requirements

- *FoodOrder must have no usable no-argument constructor — only a parameterized one that validates both fields.*
- *A blank, null, or whitespace-only studentName or dishName must be rejected at construction time.*
- *markDelivered() must print a different message the second time it's called on the same order.*

Function Signature(s)

```
public FoodOrder(String studentName, String dishName)
void markDelivered()
static void processBatch(String[][] rawOrders)
```

Examples

Input	Output
{{"Ravi","Paneer Butter Masala"}, {"","Chole Bhature"}, {"Meera"," "}, {"Divya","Veg Biryani"}}	Valid: 2 | Rejected: 2

Explanation: *The second entry has no student name, and the third has a dish name that's only whitespace — both are rejected, for two different reasons.*

HINT

Trimming a string before checking whether it's empty catches more real-world garbage input than checking for exact emptiness alone.

CONSTRAINTS

Batch size up to 500 entries.

PROBLEM 2 • **Basic**

ASAP or Scheduled — Delivery Slot Booking

Problem Statement

When a student places an order, they can either pick a specific delivery time slot, or just tap “ASAP” and let the app assign the next available default slot. The booking feature needs to support both without writing the same setup logic twice, and the app also wants to flag bookings that land in a peak-hour slot, so kitchens can staff up in advance.

Requirements

- *DeliverySlot must offer two constructors, linked through this(...) chaining — no duplicated field-setting logic between them.*
- *A slot created without an explicit time must default to "ASAP" through the chained constructor, not a separately repeated assignment.*
- *isPeakHour() must correctly flag exactly these four slot values as peak: "12:00-13:00", "13:00-14:00", "19:00-20:00", "20:00-21:00".*

Function Signature(s)

```
public DeliverySlot(String orderId, String timeSlot)
public DeliverySlot(String orderId) // chains via this(...)
boolean isPeakHour()
```

Examples

Input	Output
new DeliverySlot("ORD101", "13:00-14:00").isPeakHour()	true
new DeliverySlot("ORD102").isPeakHour() // timeSlot defaults to "ASAP"	false

Explanation: *timeSlot values are always one of a small fixed set of slot strings (or "ASAP") — no time parsing needed, just comparison.*

HINT

this(...) must be the very first statement in the constructor that uses it — and "ASAP" should only ever be typed once in your whole class.

CONSTRAINTS

timeSlot is always one of a fixed set of slot strings, or "ASAP".

PROBLEM 3 • Intermediate

Canteen Trust-Score Ranking Engine

Problem Statement

The app's homepage lists hostel-block canteens ordered by a trust score out of 5, and ties — common since scores are whole numbers — need to be broken the exact same way every time the list refreshes, or the homepage would visibly reorder itself between page loads for no reason. Design Canteen's comparison logic and a ranking function precisely enough that two runs on the same input always agree, and decide how two canteen codes that differ only in letter case should be treated in that ordering — without changing how a canteen's own code is actually stored or displayed.

Requirements

- *Canteen must resolve any field/parameter naming clash with this, and offer a second constructor that chains to the first with a sensible default trust score.*
- *The comparison method must return a signed result, following Java's own ordering convention — not a boolean.*
- *Implement the ranking sort yourself; do not call a built-in sort utility.*

Function Signature(s)

```
public Canteen(String canteenCode, String canteenName, int trustScore)
public Canteen(String canteenCode, String canteenName) // chains via this(...)
int compareTo(Canteen other)
static Canteen[] rankCanteens(Canteen[] canteens)
```

Examples

Input	Output
[Canteen("HB3-C","Spice Junction",3), Canteen("hb1-c","Grand Mess",5), Canteen("HB2-C","Southern Treats")] // 3rd defaults to trustScore 3	["hb1-c", "HB2-C", "HB3-C"]

Explanation: *Grand Mess has the highest score and ranks first outright. Spice Junction and Southern Treats are tied at 3, so the tie-break decides: work out why HB2-C sorts before HB3-C.*

HINT

Decide your tie-breaking order (score, then code, then name length) before you write a single line of compareTo() — writing the rule down first will save you from tangled if-else logic.

CONSTRAINTS

Up to 100 canteens in a single ranking call.

An $O(n^2)$ sort is perfectly fine here — correctness of the ordering matters far more than speed.

PROBLEM 4 • Intermediate

Exam-Week Surge Fee Calculator

Problem Statement

During exam week, kitchen queues back up and the app wants a fairer surge-fee model than a flat rate: the longer an order sits in the queue past the normal wait time, the higher the rate charged on the ADDITIONAL delay minutes — never retroactively on the whole delay. Rates: 0.5% of the order value per minute for minutes 1–5 of delay, 1% per minute for minutes 6–15, and 2% per minute from minute 16 onward. Separately, the calculator's configured rate should act as a minimum surge floor, which only ever applies once an order is genuinely delayed — never on an order that left the kitchen on time.

Requirements

- *The calculation rule, the field it depends on, and the class itself must each be locked against modification exactly where that protection makes sense.*
- *Negative orderValue or delayMinutes must be rejected at the point of calculation, not just at construction.*

Function Signature(s)

```
public SurgeFeeCalculator(double minimumSurgePercent)
final double calculateSurgeFee(double orderValue, int delayMinutes)
```

Examples

Input	Output
orderValue = 500, delayMinutes = 0	Rs 0.0
orderValue = 500, delayMinutes = 1	Rs 5.0
orderValue = 500, delayMinutes = 16	Rs 72.5

Explanation: *delayMinutes = 1 produces a tiered amount (Rs 2.5) smaller than the 1% floor (Rs 5.0) — work out which value actually gets charged, and why delayMinutes = 0 must never trigger the floor at all.*

Explanation: *delayMinutes = 16 crosses all three brackets — verify your bracket boundaries land exactly at minutes 5, 15, and 16.*

HINT

An order 6 minutes delayed is not “6 minutes at the minute-6-15 rate” — work out how many minutes actually fall in each bracket before multiplying anything.

CONSTRAINTS

delayMinutes will not exceed 500.

orderValue is a positive double up to roughly 10^4.

A closed-form O(1) calculation is preferred over a minute-by-minute loop.

PROBLEM 5 • Advanced

Nightly Multi-Kitchen Reconciliation Engine

Problem Statement

Every night, the app reconciles every delivery account across campus — a mix of regular student accounts and premium-member accounts, who get surge fees settled differently — and this year must also tolerate null placeholder entries left behind by a buggy sync job, without ever letting the whole run crash because of one bad entry. Build a processor that safely handles a mix of real accounts, premium accounts, and outright nulls, computing a surge fee for every account actually processed and producing a trustworthy end-of-run summary. You're given three parallel arrays as input — decide for yourself what should happen if they don't all have matching lengths, and be ready to justify that choice the way a real operations manager would.

Requirements

- *DeliveryAccount must set up any one-time, class-level state using a static block, and support both a full constructor and a provisional one that chains to it.*
- *The processor must use instanceof to decide how each account is settled, and must never throw an uncaught exception because of a null entry.*
- *The surge fee calculation must be final (reuse Problem 4's version if you built it, or a simpler flat-rate version otherwise — state which).*

Function Signature(s)

```
public DeliveryAccount(String studentId, double orderValue)
public DeliveryAccount(String studentId) // chains via this(...)
final double calculateSurgeFee(int delayMinutes)
void processAccount(DeliveryAccount account, double amount, int delayMinutes)
static void processBatch(DeliveryAccount[] accounts, double[] amounts, int[] delayMinutesArray)
```

Examples

Input	Output
accounts = {Premium("STU001",500), null, DeliveryAccount("STU002",300)} amounts = {500, 400, 300} delayMinutesArray = {10, 5, 0}	2 processed | 1 null skipped | 1 premium | 1 regular | grand total surge fees = ...

Explanation: *Index 1 is null. Decide exactly what your processor does with it, and make sure the grand total only ever reflects accounts that were genuinely processed.*

HINT

What's worse for the kitchen operations team: an error before anything runs, or a batch that quietly processes the wrong amount against the wrong student?

CONSTRAINTS

Batch size up to 10^4 accounts.

Aim for a single pass over the batch, with O(1) extra space beyond a handful of running counters.